

Procesado y análisis de datos con Python

Rocío Cruz Ramírez y José Blanco Chica

Índice

Introducción al procesamiento en Python	2
Definición de los objetivos clave	2
Fase I	3
Leer en tiempo real un log generado por una CU - El Productor	3
Identificar los mensajes con información de volumen de datos transferido en DL a nivel SDAP - Expresiones regulares	4
Guardar en un DataFrame, en tiempo real, los volúmenes de datos indicados para cada UE - El consumidor	4
Agregar cada 1 segundo y guardar en otro DataFrame los volúmenes agregados en cada UE	5
Representar en tiempo real con Dash la evolución de volúmenes agregados para cada UE	6
Cada 30 segundos guardar en un archivo Parquet la evolución de volúmenes agregados en cada UE	8
Resultados obtenidos para la fase I - Visualización en Dash	9
Fase II	9
Identificar el PLMN, PCI y RNTI de cada UE y añadir esta información en la representación de Dash	10
Resultados obtenidos para la fase II	11
Conclusiones	12

Introducción al procesamiento en Python

El proyecto se centra en el procesamiento y el análisis de datos en el sector de las telecomunicaciones, concretamente en las redes móviles de quinta generación o también 5G. Se plantea el desarrollo de una herramienta capaz de leer, procesar y representar los datos contenidos en un fichero .log que muestran la evolución del volumen de datos transmitido por varios usuarios (UE) en el sistema.

Para la implementación del proyecto se ha empleado el lenguaje de programación Python debido a su versatilidad, facilidad de uso por una sintaxis clara y sencilla y amplio sistema de librerías orientadas al análisis de datos y desarrollo de aplicaciones. Se puede definir como un lenguaje de programación de alto nivel e interpretado.

En el proyecto se hace uso de diversas librerías. Por un lado, la librería Polars se emplea para la gestión eficiente de *dataframes*, estructura de datos bidimensional compuesta por filas y columnas, donde cada columna puede contener datos de un tipo distinto. Polars ofrece alto rendimiento incluso con grandes volúmenes de información. Por otro lado, se utiliza Dash junto a Plotly para la construcción de una interfaz web que permita la visualización, en tiempo real, de los datos procesados. Otra biblioteca empleada es Asyncio que permite ejecutar varias tareas de forma concurrente.

Desde el punto de vista de las telecomunicaciones, el proyecto se sitúa en la arquitectura de redes 5G, en la que los usuarios se conectan a la red a través de las redes de acceso radio (RAN), formada por estaciones base (gNB), las cuales se comunican con el núcleo de la red (5GC). Dentro de esta arquitectura, es fundamental distinguir entre el plano de control, encargado de la señalización, y el plano de usuario, responsable de la transmisión de datos. En este trabajo, el análisis se centra en el plano de usuario, concretamente en el tráfico descendente (downlink) observado a nivel de la capa SDAP, donde se gestionan los flujos de datos asociados a la calidad de servicio.

Definición de los objetivos clave

El trabajo se centra en la lectura en tiempo real de archivos. A partir de los ficheros dados por el profesor docente del taller se puede observar un flujo básico de procesamiento en tiempo real en el que un proceso **generador** produce datos de forma periódica mientras que otro proceso **consumidor** lee dichos datos, realiza una **agregación temporal** y los representa gráficamente mediante una interfaz web (haciendo uso de **Dash** y Plotly como ya se ha explicado).

El objetivo principal del proyecto, en una primera fase, es desarrollar un sistema capaz de leer en tiempo real los logs generados por la red, identificar los eventos relevantes asociados al tráfico de datos en downlink, agrupar esta información por usuario, realizar una agregación temporal de los datos y representarlos de forma dinámica mediante una interfaz gráfica tal y como ocurre en el simulador aportado por el profesor. De esos ficheros aportados, se hará uso del fichero [log-sim.py](#), encargado de generar líneas del .log respetando los tiempos originales.

Más adelante se añadirá en una nueva fase la visualización de los parámetros PLMN, PCI y RNTI de cada UE.

Fase I

En la fase 1 se construye el fichero [log-parser.py](#) que incluye el código necesario para cumplir los objetivos planteados. Se hará uso de la programación concurrente mediante la librería Asyncio, que permite que el productor, el consumidor y el agregador puedan trabajar conjuntamente aunque eso sí, no de forma paralela, todas las tareas en un mismo hilo continuo con la diferencia de que, cuando una se encuentre esperando para escribir o leer elementos de una cola, el código pueda seguir ejecutándose con otra tarea pero pueda volver cuando ese tiempo de espera acabe.

Leer en tiempo real un log generado por una CU - El Productor

Para la lectura del fichero log, se define la función `tail_log_producer`. Está recibe la ubicación del archivo log de lectura y la cola por la cual filtrar las líneas que coincidan con la información equivalente a los mensajes SDAP.

Una vez el fichero ha sido abierto para lectura, con la función `seek` se indica al cursor que se coloque justo al final del fichero para procesar únicamente las líneas nuevas que van apareciendo. A partir de ahí, un bucle infinito que lee línea a línea y las filtra a partir de la **expresión regular** definida previamente. Más adelante se explicará con detalle qué concepto de expresión regular. Estos datos obtenidos de la línea se guardan en un diccionario y se añaden a la cola. En dicho diccionario quedan guardados el timestamp, el identificador del usuario (UE) y el tamaño, en bytes, del paquete.

En resumen, se espera a que llegue un fichero log para leer las líneas nuevas en tiempo real. Las líneas relevantes, equivalentes a los mensajes de nivel SDAP, se filtran, se guardan los datos relevantes y se envían a una cola hacia el consumidor para procesarlos.

```
while True:
    line = f.readline()

    if not line:
        await asyncio.sleep(0.05) |
        continue

    m = sdap_re.search(line)

    if m:
        await data_queue.put({
            "ts": datetime.fromisoformat(m.group("ts")),
            "ue": int(m.group("ue")),
            "pdu_len": int(m.group("pdu_len")),
        })
```

Si no se detecta una línea nueva, se espera sin bloquear el simulador.

Identificar los mensajes con información de volumen de datos transferido en DL a nivel SDAP - Expresiones regulares

El productor identifica los mensajes con información de volumen de datos mediante las expresiones regulares. Las expresiones regulares son un “lenguaje” que permite definir patrones de búsqueda sobre texto, con el objetivo de localizar, validar, extraer o sustituir cadenas de caracteres dentro de un conjunto de datos.

Una línea, dentro del fichero log, de nuestro interés por incluir la información que se busca aparece de la siguiente forma:

```
2026-01-19T08:57:06.466117 [SDAP] [D] ue=1 psi=4 QFI=1 DRB1 DL: TX PDU. QFI=1 pdu_len=1290
```

Esta línea pertenece al plano de usuario del UE y contiene la información sobre su identificador en ese plano y el tamaño en bytes.

El código debe ser capaz de identificar que ese tipo de líneas son las que se quieren filtrar, guardar y enviar a la cola hacia el consumidor. En nuestro caso, la expresión regular encargada de llevar ese propósito es:

```
sdap_re = re.compile(  
    r"^(?P<ts>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+).*"  
    r"\[SDAP\s+\].*ue=(?P<ue>\d+).*DL: TX PDU.*pdu_len=(?P<pdu_len>\d+)"  
)
```

En la imagen se observa el código de la expresión regular empleada en el simulador. La capa SDAP queda identificada con `\[SDAP\s+\]`:

- `\[SDAP` busca la palabra tal cual en la línea leída
- `\s+` busca, tras leer SDAP, uno o más espacios
- `\]` cierra

El inicio de la expresión regular (`^(?P<ts>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+)`) identifica el timestamp, más adelante se identifica al UE (`ue=(?P<ue>\d+)`) y finalmente el tamaño de paquete (`pdu_len=(?P<pdu_len>\d+)`). El uso de `\d+` indica que a continuación vienen varios dígitos numéricos y el uso de `.*` identifica cualquier texto intermedio.

Guardar en un DataFrame, en tiempo real, los volúmenes de datos indicados para cada UE - El consumidor

El consumidor es el encargado de llevar a cabo esta tarea. Es consumidor porque lee de la cola los datos que llegan del productor, extrae los datos clave (el UE y el tamaño en bytes *pdu_len*) y los guarda en una lista de diccionarios (*raw_rows*) a la vez que controla los UEs que existen en el sistema guardándolos en un conjunto de datos (*know_ues*). El tamaño en bytes de cada UE se guarda en un *buffer*.

La programación concurrente, en este caso, consiste en esperar hasta que lleguen eventos de la cola.

```

async def consumer(data_queue: asyncio.Queue):
    while True:
        event = await data_queue.get()
        # Espera hasta que haya un evento disponible en la cola
        # Cuando hay un evento, lo saca y lo guarda en la variable EVENT

        ue = event["ue"]
        pdu_len = event["pdu_len"]

        # Guarda el evento en el diccionario raw_rows
        # Esto permite tener un control de los eventos, como un historial
        raw_rows.append({
            "ts": event["ts"],
            "ue": ue,
            "pdu_len": pdu_len,
        })

        # Ahora guardamos los usuarios conocidos y los bytes recibidos
        known_ues.add(ue)
        buffer[ue] = buffer.get(ue, 0) + pdu_len # Los bytes se acumulan

        data_queue.task_done() # Evento procesado

```

Agregar cada 1 segundo y guardar en otro DataFrame los volúmenes agregados en cada UE

Para ello se emplea la función *aggregator* (o agregador en español), encargado de agregar el tráfico por UE en intervalos de tiempo de 1 segundo. En cada segundo, recorre todos los UEs conocidos del conjunto *known_ues* y saca el total de bytes de cada UE del *buffer*. Tras ello, se añaden los datos (timestamp, UE y total bytes) a otra lista de diccionarios (*agg_rows*). Al acabar, el *buffer* se resetea. De esta forma, cada segundo se van agrupando los volúmenes de datos por usuario.

Además, al acabar se convierte la lista a *dataframe* y se guardan los datos en formato Parquet. Se hace cada 30 segundos. Más adelante se tratará este apartado.

```
async def aggregator():
    last_parquet = time.time() # Guardamos el instante actual para controlar el tiempo de guardado

    while True:
        await asyncio.sleep(AGG_PERIOD) # Esperamos un segundo para hacer el siguiente ciclo

        now = datetime.now()

        # Recorremos todos los UEs
        for ue in sorted(known_ues):
            # Para cada UE, obtenemos el total de bytes acumulados en el buffer
            total_bytes = buffer.get(ue, 0)

            agg_rows.append({
                "ts": now,
                "ue": ue,
                "bytes_dl": total_bytes,
            })

        buffer.clear() # Limpiamos el buffer para, en el siguiente segundo, guardar los datos de los UEs que se han conectado

        # Cada 30 segundos guardar en un archivo Parquet la evolución de los datos
        if time.time() - last_parquet >= PARQUET_PERIOD and agg_rows:
            # Si han pasado 30 segundos desde la última vez que guardamos el archivo
            pl.DataFrame(agg_rows).write_parquet(PARQUET_FILE)
            last_parquet = time.time()
```

Se puede observar que si un UE no tiene tráfico de datos al extraerlos del *buffer*, se asigna con valor 0. Esto se hace para evitar que aparezcan líneas diagonales en el Dash.

Representar en tiempo real con Dash la evolución de volúmenes agregados para cada UE

Finalmente, se pretende observar la evolución del tráfico de datos en un gráfico web. Esa web se encuentra en la dirección IP '127.0.0.1' con el puerto 8059, ambos escritos en el código. Entrando en esa web se visualiza mediante, un gráfico de líneas, el volumen agregado para cada UE en distintos colores. Cuando un UE se desconecta, en vez de desaparecer, su tráfico se marca con valor 0 (tal y cómo se ha explicado en el apartado anterior referente al agregador. El código del Dash tiene la siguiente estructura:

```
app = Dash(__name__)

app.layout = html.Div([ # Todo lo que esté dentro de htm
    html.H2(
        "Volumen DL agregado por UE - SDAP",
        style={"textAlign": "center"}
    ),

    dcc.Graph(id="live-graph"), # LIVE-GRAPH es el ident

    # IMPORTANTE: Creamos un temporizador
    # La idea es que el temporizador se dispare cada seg
    dcc.Interval(
        id="timer", # Identificador del temporizador
        interval=1000, # Intervalo de tiempo en milisegu
        n_intervals=0, # Contador de intervalos, se incr
    ),
])
```

En esta parte del código, se inicializa el Dash y se asigna un título y un gráfico a la página web. Además, también es importante señalar el papel del temporizador, que cada 1 segundo se encargará de actualizar el gráfico. Ese tiempo definido de 1 segundo (o 1000ms en el código) coincide con el tiempo de agregación de datos.

```
@app.callback(
    Output("live-graph", "figure"),
    Input("timer", "n_intervals")
) # Cada vez que el temporizador se dispare, se llamará a la
def update_graph(_):
    fig = go.Figure()

    if not agg_rows: # Cuando no hay datos agregados, útil pa
        fig.update_layout(
            title="Esperando datos SDAP DL...",
            xaxis_title="Tiempo",
            yaxis_title="Bytes DL/s",
        )
    return fig

# A partir de Polars, generamos un dataframe a partir de
# Necesario para hacer el gráfico con Plotly
df = pl.DataFrame(agg_rows)
```

La función encargada de actualizar el gráfico es `@app.callback` y lo hace cada 1 segundo, tal y como estaba especificado previamente. Si aún no hay datos agregados en la lista `agg_rows` (como ocurre al principio de la simulación), mostrará en pantalla el mensaje siguiente: “Esperando datos SDAP DL...”. Sin embargo, en el momento en el que estos empiezan a llegar, se crea un *dataframe* de Polars para filtrarlos y organizarlos fácilmente.


```
# Recorremos cada UE que hay en el dataframe
for ue in df["ue"].unique().sort():
    ue_int = int(ue)
    df_ue = df.filter(pl.col("ue") == ue_int)
    # Aquí le hemos pasado un filtro al dataframe para quedarnos solo con la UE
    # Añadimos ahora a la gráfica
    fig.add_trace(go.Scatter(
        x=df_ue["ts"].to_list(),
        y=df_ue["bytes_dl"].to_list(),
        mode="lines+markers", # Líneas y puntos, podríamos haber usado solo líneas
        name=f"UE {ue_int}",
    ))

fig.update_layout(
    title={
        "text": "Volumen DL agregado cada 1 segundo por UE",
        "x": 0.5,
        "xanchor": "center",
    },
    xaxis_title="Tiempo",
    yaxis_title="Bytes DL/s",
)

return fig
```

La última parte del código referente al Dash es la encargada de leer la información contenida sobre cada UE en el *dataframe* y representarla en el gráfico. De esta manera, se consigue representar en tiempo real la evolución de los datos por cada UE en el *downlink*, empleando el temporizador que permite la actualización de los datos que previamente han sido agregados cada 1 segundo.

Cada 30 segundos guardar en un archivo Parquet la evolución de volúmenes agregados en cada UE

Un archivo Parquet es un formato de almacenamiento de datos en columnas diseñado para guardar grandes volúmenes de información de forma eficiente y rápida para consultas analíticas. En el proyecto, los datos de timestamp, UE y tamaño en bytes, que se guardan cada segundo en la lista *agg_rows* empleada para la visualización en tiempo real del tráfico, se guardan cada 30 segundos en un archivo Parquet.

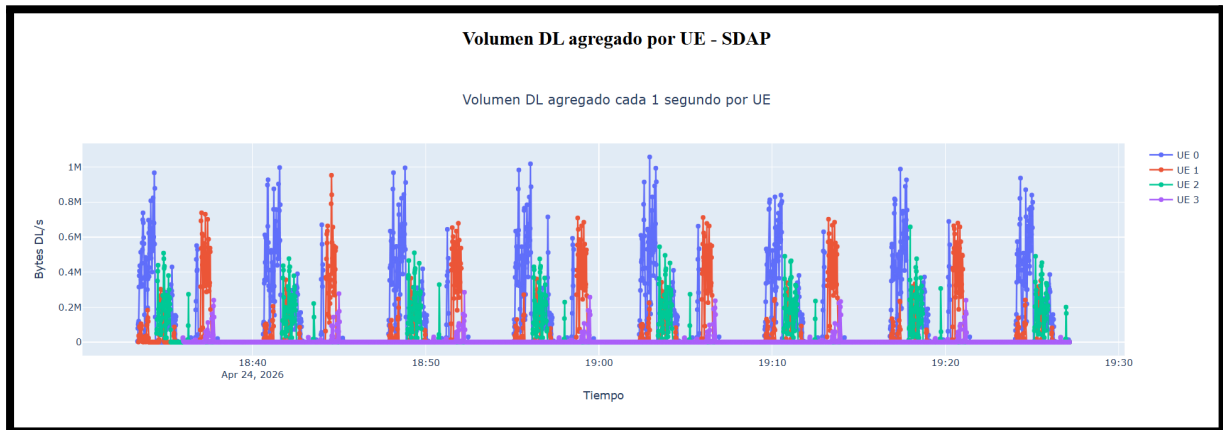
```
# Cada 30 segundos guardar en un archivo Parquet la evolución de volúmenes agregados
if time.time() - last_parquet >= PARQUET_PERIOD and agg_rows:
    # Si han pasado 30 segundos desde la última vez que guardamos el parquet
    pl.DataFrame(agg_rows).write_parquet(PARQUET_FILE)
    last_parquet = time.time()
```

La variable *last_parquet* guarda el último instante en el que se generó el anterior archivo parquet, y si la diferencia entre este y el instante actual (medido con *time.time()*) es superior o igual a 30 segundos

(determinado por PARQUET_PERIOD) y, a la vez, hay información en *agg_rows*, entonces se genera un nuevo archivo Parquet a partir del *dataframe*.

Resultados obtenidos para la fase I - Visualización en Dash

La siguiente imagen muestra cómo ha quedado la interfaz web del Dash y el Plotly. Cada identificador UE aparece de un color diferente, y cuando el UE se desconecta de la red su tráfico se establece a cero.



El gráfico muestra el tamaño en bytes del volumen de datos en el *downlink* en cada segundo. Se observa que el UE 0 (azul) presenta grandes volúmenes de datos en ciertos instantes de tiempo, mientras que el UE 3 es el que menos tráfico presenta.

Fase II

Partiendo del código anterior, se pretende visualizar en la interfaz web junto al gráfico del volumen de datos la información acerca del PLMN, el PCI y el RNTI de cada UE registrado.

- **Public Land Mobile Network (PLMN):** Identificador de una red móvil pública, indicando el operador al que pertenece una red. Está compuesto por el MCC y por el MNC
- **Physical Cell Identity (PCI):** Identificador físico de una celda radio. Tiene un rango de 0 a 503, y su valor es reutilizable. Permite al UE identificar y sincronizarse con la celda.
- **Radio Network Temporary Identifier (RNTI):** Identificador temporal único asignado a un UE dentro de una celda

Dicha información se encuentra en el plano de control del CU.

```
2026-01-19T08:56:15.341870 [CU-UEMNG] [I] ue=6 du_index=0 plmn=21405 gnb_du_id=5 pci=501  
rnti=0x4605 pcell_index=0: Created new CU-CP UE
```

El usuario tiene distinto identificador UE según el plano sea de control (CP) o de usuario (UP). Si recuerda la imagen de apartado “Identificar los mensajes con información de volumen de datos transferido en DL a nivel SDAP - Expresiones regulares” de la fase I, podrá observar que el identificador del UE para el CP es 1 (ue=1). La solución para tener un único identificador del UE en

ambos planos es la variable `cu_cp_ue` que es igual a la variable `cu_up_ue` y que denominaremos identificador intermedio.

```
2026-01-19T08:56:15.789426 [CU-UP-E1] [I] Tx PDU ue=1 cu_cp_ue=5 cu_up_ue=5: BearerContextSetupResponse
2026-01-19T08:56:15.789494 [CU-CP-E1] [I] Rx PDU ue=6 cu_cp_ue=5 cu_up_ue=5: BearerContextSetupResponse
```



Relación entre los identificadores 'ue' de CP y 'ue' de UP para un UE

Así, se podrá identificar al usuario en ambos planos a partir de dicha variable.

Identificar el PLMN, PCI y RNTI de cada UE y añadir esta información en la representación de Dash

En la fase I, se mapea la línea con la información de volumen de datos transferido en DL a nivel SDAP, contenida en el plano de usuario. Por tanto, se añaden tres nuevas expresiones regulares: una para mapear el valor del identificador intermedio `cu_up_ue` y con ello, obtener `cu_cp_ue`, a partir del identificador UE de la primera expresión regular, una segunda para obtener el identificador UE en el plano de control con el identificador `cu_cp_ue` y una tercera para extraer la información relativa al PLMN, PCI y RNTI.

```
# Expresión regular para extraer información de UE creada en CU-CP (plano de control)
cp_info_re = re.compile(
    r"\[CU-UEMNG\].*ue=(?P<cp_ue>\d+).*plmn=(?P<plmn>\d+).*"
    r"pci=(?P<pci>\d+).*rnti=(?P<rnti>0x[0-9a-fA-F]+)"
)
# Expresión regular para mapear UE CP -> cu_cp_ue
cp_to_e1_re = re.compile(
    r"\[CU-CP-E1\].*ue=(?P<cp_ue>\d+).*cu_cp_ue=(?P<cu_cp_ue>\d+)"
)
# Expresión regular para mapear UE UP -> cu_cp_ue
up_to_e1_re = re.compile(
    r"\[CU-UP-E1\].*ue=(?P<up_ue>\d+).*cu_cp_ue=(?P<cu_cp_ue>\d+)"
)
```

También se crean tres nuevas estructuras para guardar la información mapeada cómodamente y que el UE del plano de usuario tenga la información relativa del plano de control.

```
# Estructuras nuevas de la fase 2
cp_ue_info = {} # Mapea UE del plano de control (CP) a su información (PLMN, PCI, RNTI)
cu_cp_ue_to_cp_ue = {} # Mapea UE del plano de control (CP) a UE del plano de usuario (UP)
up_ue_info = {} # Mapea UE del plano de usuario (UP) a su información (PLMN, PCI, RNTI) obtenida a
```

- `cp_ue_info`: lista de diccionarios con la información del UE del plano de control
- `cu_cp_ue_to_cp_ue`: lista con los identificadores intermedios de cada UE del plano de control
- `up_ue_info`: lista con la información del plano de control que tiene cada UE del plano de usuario

El productor y el consumidor seguirán funcionando igual que en la fase anterior, con la diferencia de que el productor leerá y generará nuevos eventos (además de los referentes al SDAP) con las expresiones regulares añadidas y el consumidor se encargará de unir las piezas sueltas. Para ello, el consumidor extrae de la cola la información de interés,

```
if event_type == "cp_info":
    cp_ue_info[event["cp_ue"]] = {
        "plmn": event["plmn"],
        "pci": event["pci"],
        "rnti": event["rnti"],
    }
```

Tras ello, guarda la relación entre los dos identificadores, el intermedio *cu_cp_ue* y el UE real en el plano de control *cp_ue*,

```
elif event_type == "cp_map":
    cu_cp_ue_to_cp_ue[event["cu_cp_ue"]] = event["cp_ue"]
```

Para finalmente, extraer los identificadores en el plano de usuario *up_ue* y el identificador intermedio respectivo, buscar que *cp_ue* pertenece a ese identificador intermedio y obtener la información guardada a ese UE (PLMN, PCI y RNTI). La lista final de salida es *up_ue_info*.

```
elif event_type == "up_map":
    up_ue = event["up_ue"]
    cu_cp_ue = event["cu_cp_ue"]

    cp_ue = cu_cp_ue_to_cp_ue.get(cu_cp_ue)

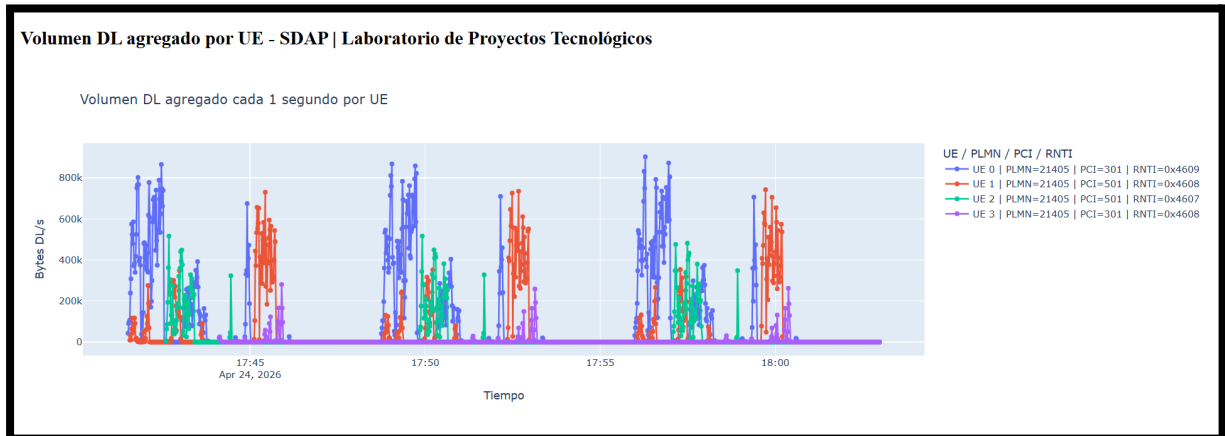
    if cp_ue in cp_ue_info and up_ue not in up_ue_info:
        up_ue_info[up_ue] = cp_ue_info[cp_ue]
```

Por último el agregador extraerá la información tanto de la lista *up_ue_info* como del *buffer* con el total de bytes para su representación en el Dash.

Resultados obtenidos para la fase II

Los resultados en la interfaz web son iguales a los obtenidos en la fase anterior pero con la información acerca del PCI, el RNTI y el PLMN de cada UE. Cada UE tiene su propio identificador

RNTI. Los UE 0 y 3 están en la misma celda y los UE 1 y 2 en otra diferente, debido a los valores que presentan del PCI. El PLMN no cambia porque todos pertenecen a la misma red.



Conclusiones

El proyecto ha servido como primer contacto con el análisis de datos en Polars y con Parquets, la programación en Python con bibliotecas como Dash o Plotly y comprender el concepto de productor-consumidor-agregador propios de una *pipeline*. Además ha permitido entender y practicar conceptos como la programación concurrente mediante la biblioteca Asyncio. Por otro lado, también ha servido para recordar conceptos de 5G y New Radio.

Nos encontramos muy satisfechos con el trabajo y los objetivos logrados.